SAGE

A Small, Vendor-Neutral Handshake for Participating Media Software

Consolidated White Paper v1.07

Design revision after feasibility testing and participant-handshake discussion

Status: design revision with a v0.02 Core draft. The repository implementation remains the authoritative record of what is currently implemented. The v0.01 wire format is preserved as historical documentation only; the active runtime is v0.02-only.

SAGE is deliberately small. It is a handshake, not a complete provenance ledger, authenticity service, forensic engine, or vendor certification system.

1. Executive Summary

SAGE provides a compact, open signal that a participating system interacted with an asset. Its only globally standardized semantic is PARTICIPANT_ID. A participant may be an AI model, image editor, camera system, transcoder, CMS, mobile application, or other conforming implementation.

The protocol does not claim that an asset is authentic, that a chain is complete, or that an unmarked asset was created by a person. It reports surviving SAGE evidence and leaves interpretation to the receiving application.

The preferred cooperating workflow is:

- Read any recoverable SAGE participant chain before processing.

- Perform the participant's operation.

- Replace the participant's existing entry if present, then append the current entry.

- Write a fresh SAGE record to the output media.

- Locally decode and self-check the result.

This allows a tag-aware AI, PHP/GD service, JavaScript editor, Dart application, camera pipeline, or transcoder to preserve logical participation even when the original pixels and ordinary metadata are replaced.

2. Design Principles

- Handshake first: standardize only what independent implementations must agree on.

- Participant-owned meaning: optional extension values are opaque to SAGE.

- Evidence, not authority: detection and registry lookup do not authenticate media.

- Cooperation over persistence: conforming processors re-tag outputs after transformation.

- Offline by default: network services are optional enrichment, never a decoding prerequisite.

- Language neutrality: shared vectors matter more than a preferred runtime.

3. Core Handshake Model

The new design replaces the AI-specific identity model with a participant model. PARTICIPANT_ID is the sole globally meaningful value. It identifies the participating implementation namespace, not necessarily a company and not necessarily a single product.

A participant may register separate identifiers for products, versions, models, capture systems, or deployment environments. Registration is discovery only and is not certification.

3.1 Participant entry

The proposed participant entry is:

```
PARTICIPANT_ENTRY { PARTICIPANT_ID, EXT_DATA_1, EXT_DATA_2, EXT_DATA_3 }
```

Only PARTICIPANT_ID has standardized meaning. The three extension slots are optional, bounded, participant-scoped opaque values. They may contain a participant's user reference, timestamp, capture ID, generation reference, copyright claim, workflow ID, or nothing at all. SAGE transports them but does not vouch for their truth.

3.2 Unique participant chain

A PARTICIPANT_ID may appear at most once in the active chain. SAGE is therefore not an event-by-event ledger. It preserves the represented participants and their relative most-recent SAGE-recorded order.

```
Example: A -> B -> C -> A becomes B -> C -> A. The prior A entry is removed or replaced, and the
current A entry is appended.
```

Chain position is relative last-touch order only. It is not authenticated wall-clock chronology and does not prove that no unrepresented system touched the asset.

3.3 Timestamp policy

SAGE does not require a timestamp. Incorrect clocks, timezone normalization, falsifiable values, and unnecessary payload cost make a mandatory timestamp undesirable. A participant may place a timestamp in an extension slot under its own meaning.

4. Proposed Canonical Representation

The v0.02 wire syntax below is the current implementation draft. It must remain versioned and should be frozen with independent conformance vectors before promotion beyond the experimental series.

```
SAGE/0.02|||||...
```

A normative revision should define UTF-8 encoding, identifier bounds, extension presence and length limits, delimiter escaping or binary framing, canonical equality, malformed-record behavior, and compatibility policy for later versions.

The active parser accepts v0.02 only and rejects earlier or unknown versions explicitly rather than silently migrating or misinterpreting them. Earlier formats remain available as historical specifications and repository history.

4.1 Update operation

Input condition	Required update
No valid record	Create a new chain only when the caller supplies the source/context required by the frozen revision.
Current participant absent	Append a new participant entry.
Current participant present	Remove or replace its old entry, then append the current entry.
Conflicting valid records	Fail closed and preserve diagnostics; do not silently select one.
Damaged or invalid prior evidence	Follow explicit profile policy; never present an unrecoverable chain as complete.

4.2 Equality

Canonical equality compares parsed logical participant entries and extension values, not raw byte strings. Serialization must be deterministic so independent implementations produce identical vectors.

5. Media Evidence Channels

SAGE may expose multiple evidence channels with different failure characteristics. None is authoritative by itself.

Channel	Strength	Typical failure
Metadata	Fast and exact when preserved	Dropped by image libraries, social platforms, screenshots, and re-encoding
Concealed	Unobtrusive and potentially transform-tolerant	Pixel changes, crop, resize, recompression, or profile mismatch
Visible mini-SAGE	Human-visible and screenshot-tolerant	Crop, paint-over, copying, or stale visible marker

5.1 Metadata

Metadata is the preferred cooperative transport when the media format supports it. A participant must parse before transformation and write after transformation. A GD-style pipeline that recreates a PNG must explicitly re-add SAGE metadata because the image library may discard ancillary chunks.

5.2 Concealed transport

Concealed placement, redundancy, checksums, error correction, capacity, and recovery thresholds belong to media profiles, not Core. The current repository's PNG concealed layer is experimental and has demonstrated exact recovery on untouched raster passes but poor recovery under color changes, filtering, cropping, and other transforms.

5.3 Visible mini-SAGE

An optional visible marker may encode the SAGE version and final recorded PARTICIPANT_ID in a compact profile-defined symbol. It must not change Core semantics, and a decoder must never treat it as proof that hidden or metadata evidence still exists.

6. Evidence and Decoder Semantics

A decoder reports surviving evidence, not a platform verdict. The core outcomes remain PRESENT, NOT_DETECTED, DAMAGED, and CONFLICT, with candidate records and diagnostics preserved.

Observation	Interpretation
PRESENT	At least one valid SAGE participant record was recovered.
NOT_DETECTED	No valid record was recovered. This does not mean human-created, authentic, or untouched.
DAMAGED	SAGE-like or partial evidence exists but no complete usable result is available.
CONFLICT	Independent valid evidence channels or candidates disagree; preserve all candidates.

A valid record may identify represented participants and the final recorded participant. It cannot guarantee that every system involved is represented or that the final participant was literally the last software to touch the asset.

Recommended user-facing wording is 'Final recorded participant' or 'Latest SAGE participant', not 'Last edited by.'

6.1 Cooperative processing

A conforming participant should not treat a missing tag as proof that the input is unmarked in the world. It should record only what it knows from local evidence and caller policy. When valid prior evidence exists, it should preserve the logical chain under the unique-participant rule.

6.2 Non-cooperative processing

SAGE cannot require unrelated software to preserve metadata or concealed pixels. Loss of evidence during such processing is an expected limitation, not evidence that the asset lacked prior participation.

7. Registry and Optional Enrichment

The registry is a directory for participant discovery. It is not a provenance database, custody service, certification authority, or requirement for local decoding.

A minimal public record may contain:

Field	Meaning
PARTICIPANT_ID	Stable public identifier used in SAGE records
DISPLAY_NAME / VENDOR_NAME	Human-readable participant label
PUBLIC_URL	Participant-controlled information or documentation URL
STATUS	Small administrative state such as active, retired, or revoked

7.1 Operating modes

- OFFLINE: return participant IDs only; fully valid and default-safe.

- CACHED: resolve against a local snapshot and surface snapshot age.

- LIVE: perform explicit read-only HTTPS lookup when caller policy allows it.

Anonymous lookup should remain available. Optional API keys may increase rate limits but must not expose richer participant data or stronger provenance meaning. Keys, account IDs, secrets, and service telemetry must never enter media records.

Registry failures must not convert locally recovered PRESENT evidence into NOT_DETECTED or DAMAGED.

8. Language-Neutral Adoption Plan

SAGE should demonstrate interoperability across languages rather than require a framework or vendor runtime. Libraries should remain separable from application UI and business logic.

8.1 Practical reference layout

```
/reference/{language} contains small Core/Profile implementations. /examples contains applications
such as a PHP/JavaScript web editor or Flutter/Dart editor. Shared vectors remain the behavioral
oracle; Python is not normative merely because it is first.
```

8.2 Recommended first ports

JavaScript and PHP provide the highest immediate adoption value for browser editors, Node services, CMS systems, and GD-based pipelines. Dart is the next valuable port for mobile and Flutter demonstrations. C or Rust can follow as a native interoperability baseline.

8.3 Cross-language evidence

- PHP encode -> JavaScript decode

- JavaScript encode -> Dart decode

- Dart encode -> Python decode

- Python encode -> PHP decode

Malformed, damaged, conflicting, and extension-bearing vectors must be shared as well as happy-path vectors.

9. Threat Model and Non-Goals

SAGE records are claims and surviving evidence carried by an asset. Without independent authentication, a participant ID can be forged, copied, replayed, or removed. A visible marker can be copied without the original hidden evidence. A registry lookup confirms only that an ID maps to a public registry record.

SAGE does not:

- Prove that a provider performed an operation.

- Prove that an asset is genuine, safe, legal, or unaltered.

- Prove that an unmarked asset is human-created.

- Guarantee complete history or custody.

- Require prompts, account identities, IP addresses, or private generation data.

- Define moderation, labeling, ranking, or access decisions.

Future cryptographic signatures, authenticated namespaces, or external forensic systems may add assurance, but they should remain optional extensions until a concrete interoperability need is established.

10. Current Repository Status

The repository currently contains a Python-first v0.02 participant-handshake implementation with Core parsing, PNG metadata transport, an experimental concealed PNG profile, CLI commands, conformance vectors, transformation fixtures, and CI. Earlier v0.01 compatibility code has been removed from the active runtime; its documents and commits remain historical reference material.

Empirical testing established that the metadata layer is reliable for intact PNG workflows, while the present concealed prototype is not robust under general filtering, background changes, crop, resize, or lossy transformation. This finding supports the cooperative re-tagging model and prevents unsupported production claims.

The v0.02 Core draft now uses PARTICIPANT_ID, three optional opaque extension slots, and unique-participant reorder-on-update semantics. Cross-language implementations and final compatibility policy remain future work.

11. Implementation Roadmap

Milestone	Deliverable
A. Normative Core	Freeze PARTICIPANT_ENTRY syntax, unique-chain update rule, extension bounds, equality, and version compatibility.
B. Python oracle	Harden the v0.02 Core, profiles, CLI, vectors, and tests; keep earlier formats historical and explicitly rejected by the active parser.
C. PHP and JavaScript	Implement independent metadata/Core libraries and cross-decode shared vectors.
D. Visible profile	Prototype optional marker, scan reliability, disagreement diagnostics, and copy/crop behavior.
E. Dart	Add a Flutter-friendly reference library and cross-language fixtures.
F. Registry	Add optional anonymous lookup, snapshots, mirrors, and rate-limit behavior outside Core.
G. Concealed research	Replace the current prototype only after bootstrap, spatial placement, redundancy, ECC hooks, and measured recovery thresholds are demonstrated.

12. Promotion Criteria

- Frozen compatible Core and media-profile versions.
- Independent implementations that cross-decode one another.
- Published deterministic vectors, including malformed and conflict cases.
- Measured capacity and transformation behavior.
- Documented failure modes and explicit trust boundaries.
- No production-strength claims without supporting evidence.

Appendix A. Cooperative Processing Examples

A tag-aware image editor:

```
read_metadata(input) -> participant chain
transform(input) -> output pixels
update_participant(chain, EDITOR_ID, extension values)
write_metadata(output, updated chain)
write_concealed(output, updated chain)
decode(output) -> self-check
```

A PHP/GD service may lose all incoming PNG ancillary chunks during image recreation. That does not invalidate the cooperative model; it makes explicit parse-before-transform and write-after-transform integration necessary.

A participating AI may receive a chain A -> B, generate a new image, and output B -> AI_C. The new pixels need not retain the old hidden signal because the logical evidence is intentionally re-established by the cooperating participant.

Appendix B. Minimal Registry Response

```
{ "participant_id": "004821", "display_name": "Example Image Tool", "public_url":
"https://vendor.example/sage", "status": "active" }
```

The response maps an opaque identifier to public information. It does not authenticate the asset, extension values, chain completeness, or participant claims.

Appendix C. Closing Principle

SAGE is the handshake, not the conversation. It identifies participating systems while leaving participant-owned detail, registry enrichment, application policy, and forensic investigation outside the smallest interoperable core.